

EEE3095/6S Final Exam Solutions

November 2023

Duration: 3 hours

Total: 150 marks

Empl ID: _____

Instructions:

- This is a closed-book exam.
- There are **two** sections (Parts A and B) to this exam.
- You must use a pen to complete the exam – pencil will only be marked for drawings. Values indicated on drawings must be written in pen.
- Please show all your working clearly – your method and approach matter.
- Keep all your answers within the allocated blocks; anything outside of these blocks may not be marked.
- Two reference sheets appear at the end of this paper.

Question	Available Marks	Grade
1	10	
2	15	
3	20	
4	15	
5	15	
6	12	
7	16	
8	16	
9	15	
10	16	
Total:	150	

Part AEmpl ID:

--	--	--	--	--	--	--

Question:	1	2	3	4	5	Total
Points:	10	15	20	15	15	75
Score:						

Question 1: Multiple Choice [10 marks]

- a. If we have a real-time system that can survive task failures as long as the missed deadlines are infrequent, what kind of system is this said to be? (2)
- ☐ Real real-time
☒ **Firm real-time**
☐ Soft real-time
☐ Hard real-time
- b. According to the Brisbane definition, an embedded system is described as: (2)
- ☐ an information processing system embedded into a larger product
☐ a system that integrates computation with physical dynamics
☒ **a task-specific computer, which is built into a larger system for the purpose of controlling and monitoring the larger system**
☐ a computer system in which a mechanism is controlled or monitored by computer-based algorithms
- c. In a 4-bit system, what would be the base-10 value of 0b1001 in a signed representation? (2)
- ☐ 7
☐ -9
☐ 9
☒ **-7**
- d. A CISC processor is restricted to only working on operands that are stored in registers. True or false? (2)
- ☐ True ☒ **False**
- e. RS-232 is an asynchronous serial signaling standard that uses the voltage range (+3 V to +15 V) as a logic 1 and (-3 V to -15 V) as a logic 0. True or false? (2)
- ☐ True ☒ **False**

Question 2: ARM Processing [15 marks]

- a. Describe the term "development environment" in the context of both PC and embedded systems development, and identify at least two differences between them. (4)

Solution: The **PC development environment** is well defined and uses standard tools, allowing for easy toolchain configuration, debugging and library access.

The **embedded systems development environment** is generally not as well defined, as the tools depend on the software vendor, the hardware vendor, peripherals used, and other factors. The toolchain is also usually built from scratch, debugging requires special techniques, and there are not always support libraries available for the target platform.

- b. If we have access to ARM, AVR and PIC microprocessors, which would be most suited to each of the following devices? Give a brief reason for each answer. (4)
1. Apple iPhone
 2. ABS braking system
 3. Weather sensing device
 4. Burglar alarm

Solution: Open-ended question that will be marked based on the student's reasoning, but generally:

1. Apple iPhone: ARM for the power and efficiency required by a modern smartphone; in fact, Apple *does* use ARM processors for their devices nowadays
2. ABS braking system: Use AVR (or ARM) since it is quite a complex system; ARM may be slightly overkill for this, but we need a fast processor to keep up with the timing required by a braking system
3. Weather sensing device: PIC is likely sufficient as we do not need high speed and response times
4. Burglar alarm: PIC, assuming there is not much signal processing going on and it is a fairly basic system; otherwise potentially AVR for more advanced functionality



c. C is a very popular, powerful and flexible programming language that is often called a "portable Assembly language". Answer the following questions:

(i) In what way is C seen as "portable Assembly" compared to, for example, Python or Java?

(2)

Solution: C offers a very low level of abstraction compared to Java/Python/etc. and is known for being extremely close to the hardware, offering features such as direct memory manipulation.

- (ii) Why is C generally preferred over C++ for embedded systems development? Provide two reasons. (2)

Solution: Any two of the following:

- C++ has not seen as much widespread use in embedded systems development compared to standard C
- C++ contains a lot of baggage and language constructs, which can cause bloat
- C++ programs are (depending on style) less efficient than C programs
- C++ does not have the same wide compiler support for embedded system platforms

- (iii) What would happen if the following command was successfully run in a terminal? (3)

```
gcc -O1 main.c -o myprog
```

Solution: This would:

- compile the main.c file using the GCC compiler
- store the output executable binary in a file called "myprog"
- apply compiler optimisation level 1 (basic optimisations that do not take too long)

Question 3: Embedded Communications [20 marks]

a. RS-232 is known to have some limitations compared to (the newer) RS-485.

(i) List two limitations associated with RS-232.

(2)

Solution: Any two of the following:

- Supports only one transmitter and one receiver on a communication bus
- Data transfer can only be done for distances less than 15 m
- Performance degrades quickly when there is noise present
- Rather limited speed of data transfer (especially for lengths beyond greater than around 1 m)

(ii) List two advantages associated with RS-485.

(2)

Solution: Any two of the following:

- Can operate over a maximum of around 1200 m
- Twisted cables used to pick up less noise
- Higher maximum data transmission rate
- More drivers and receivers can be connected on one line simultaneously (up to 32 of each instead of one of each)

b. If we have an 8-bit data word 0x6A that needs to be transmitted with a parity bit appended, what would be the final 9-bit data word that is transmitted for both odd and even parity?

(2)

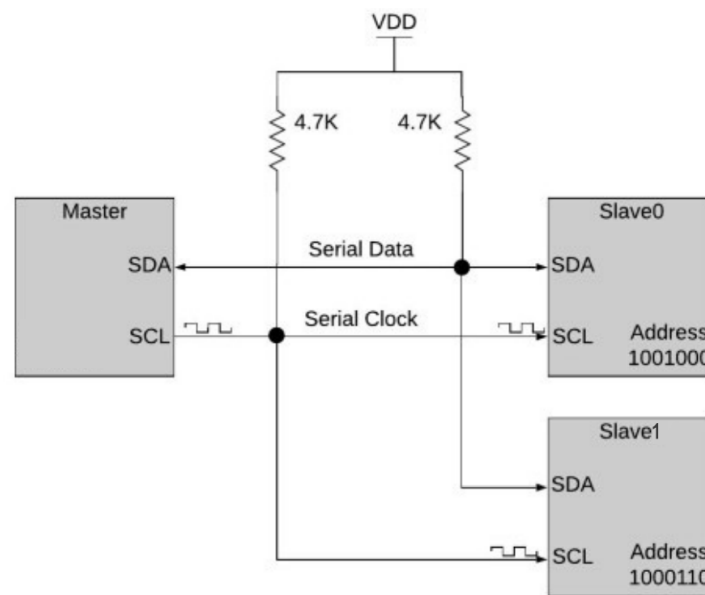
Solution: The 8-bit data is 0x6A, which corresponds to 0b01101010; this contains a total of 4 ones, so:

- for odd parity, the full 9-bit data would be 0b011010101 (a one is appended for a total of 5 ones)
- for even parity, the full 9-bit data would be 0b011010100 (a zero is appended for a total of 4 ones)

- c. Draw a labelled diagram of the physical I2C interface for one master and two slave devices, including all signal lines and slave addresses, and then briefly explain each aspect of your illustration. (8)

Solution: The figure below shows the physical I2C interface for a single master and two slave devices, including the pull-up resistors (values are not crucial) and the SCL and SDA lines. The two slave devices can be assigned any 7-bit addresses as long as they are different from one another.

VCC (or VDD) provides the input voltage to the bus, and the pull-up resistors are used to set the signal lines to be active-low. SDA (Serial Data) is used to transmit data between the master and slave devices (in either direction), while SCL (Serial Clock) is used for synchronisation purposes as a common clock signal shared between all the devices.



d. The following sub-questions relate to the SPI communication protocol.

- (i) The four SPI Modes are dependent on the Clock Phase (CPHA) and Clock Polarity (CPOL). Draw a table to illustrate how these two quantities vary based on the configured SPI Mode, and include a very brief description of each of the two terms. (4)

Solution: Table below.

SPI Mode	Clock Polarity [CPOL] Clk level when idle	Clock Phase [CPHA] Sample on first or second edge
0	0	0
1	0	1
2	1	0
3	1	1

- (ii) SPI allows only one master device to be connected, but multiple slave devices can be used. Briefly explain how we could connect up multiple *independent* slaves using the same SPI bus. (2)

Solution: This is a typical SPI bus configuration that uses dedicated Slave Select (SS) lines, e.g., for one master device and three independent slaves. In this configuration, each new slave requires an additional dedicated SS line for individual control, and when data needs to be transmitted, the selected slave's SS line will be set low to begin transfer.

Question 4: ARM Assembly [15 marks]

The following questions relate to ARM Assembly code being run on a 32-bit ARM processor.

- a. You are given the ARM assembly code below. Answer the following questions.

```
myfunc: @ void myfunc (int input); modifies a global variable myglob
        cmp r0, #0
        rsbmi r0, r0, #0
        ldr r1, =myglob @ r1 = address of myglob
loop:
        ldr r2, [r1] @ get value of myglob
        sub r2, r2, r0
        sub r2, r2, r0
        sub r2, r2, r0
        sub r2, r2, r0
        cmp r2, #0
        bgt loop
        str r2, [r1]
        ldr r2, [r1]
        bx lr @ return from function myfunc
```

- (i) Briefly explain what the `rsbmi` instruction would do in this program. (2)

Solution: The `rsb` instruction would normally apply a reverse subtraction, so in this case, the operation `r0 = 0 - r0` would be applied (essentially changing the negative `r0` value into a positive value). With the `mi` suffix appended, the `rsbmi` instruction would thus only execute the aforementioned reverse subtraction if the previous `cmp` result was negative (i.e., the Negative flag was set in the status register and thus `N = 1`).

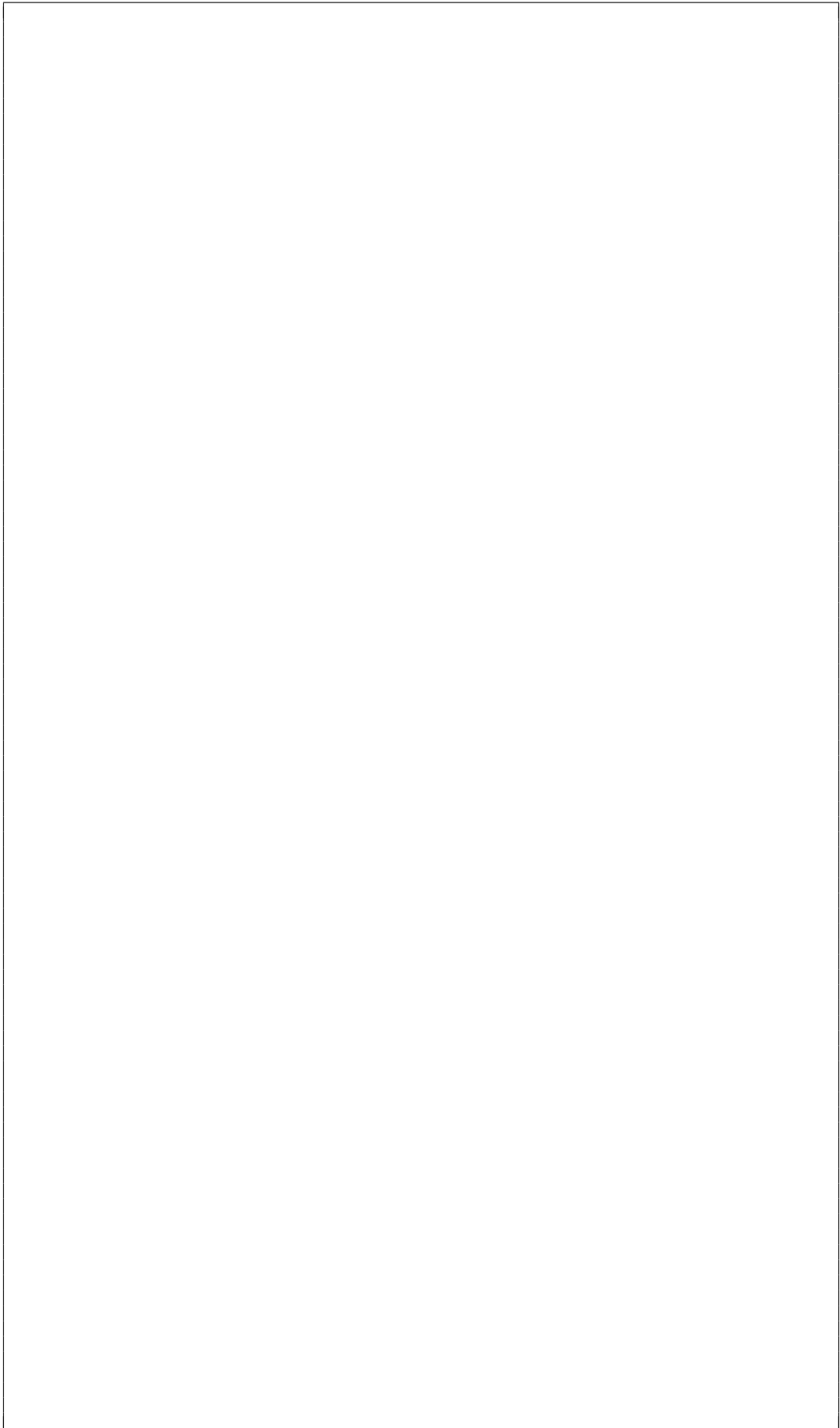
- (ii) Explain how you could optimise this program; you could do this by rewriting the optimised version of the full code and commenting on the reason for the changes, or by simply explaining each aspect of what you would change in the code (and why). (6)

Solution: A few things could be done here:

- Move `ldr r2, [r1]` from the first line of `loop` to the end of `myfunc` instead; as `loop` iterates, `r1` is not changing so there is no need to load from it on every loop
- Multiply `r0` by 4 instead of doing 4 `sub` operations; so do: `mul r0, r0, #4` and then one `sub r2, r2, r0` to reduce number of instructions in the loop
- Remove the `ldr r2, [r1]` instruction near the end of `loop`

If writing the final code, it should be something like:

```
myfunc: @ void myfunc (int input); modifies a global variable myglob
        cmp r0, #0
        rsbmi r0, r0, #0
        ldr r1, =myglob @ r1 = address of myglob
        ldr r2, [r1] @ get value of myglob once-off
        mul r0, r0, #4 @ multiply "input" r0 by 4 for the subtraction
loop:
        sub r2, r2, r0 @ reduced number of subtractions
        cmp r2, #0
        bgt loop
        str r2, [r1]
        @ removed the ldr
        bx lr @ return from function myfunc
```



- b. You are asked by a client to implement the following mathematical operation in ARM Assembly:

(7)

$$r0 = ((r1 * (r2 + 4))/8) - 4$$

If the result is negative, the value should also be squared. Write an appropriate piece of Assembly code that will meet the client's requirements, and show all reasoning using appropriate Assembly comments in your code. At the end of your code, include a `bx lr` instruction to return to a calling function.

Solution: See below:

```
add r2, r2, #4 @ r2 = r2 + 4
mul r1, r1, r2 @ r1 = r1 * r2
asr r1, r1, #3 @ r1 = r1/23 = r1/8
subs r0, r1, #4 @ r1 = r1 * r2; need "s" to set flags
bge notneg @ branch to "notneg" if r0 >= 0
mul r0, r0, r0 @ square r0 if branch not taken
notneg:
    bx lr
```

Question 5: Embedded Design [15 marks]

- a. A stack works by taking items and piling them one on top of the other. In computing, these items would be pieces of data and we would typically have two primary functions to interact with the stack, namely:

(6)

1. **push(x)**: adds the object/value **x** to the top of the stack
2. **x = pop()**: removes the topmost object/value from the stack and return it in **x**

Write code to show how you would implement this in the C programming language. Use the unsigned int data type for your stack elements and a maximum stack size of 1000.

Solution: Code below:

```
// Set maximum size of stack
#define SIZE 1000
unsigned int stack [SIZE];
unsigned int top = 0; // First place to put the next item

// Place item on top of stack
void push (unsigned int x) {
    if (top < SIZE) {
        stack[top] = x;
        top = top + 1;
    }
}

// Remove item from top of stack
unsigned int pop () {
    if (top > 0) {
        top = top - 1;
        return stack[top];
    }
}
```

- b. You are asked to develop embedded software (for your STM board) that achieves the following: (9)
- Sample the value from potentiometer POT0; assume it has been configured in 12-bit mode
 - Output the current value being sampled (from POT0) to the LCD screen on your STM board
 - If the ADC reading is at or above the halfway point (i.e., half the maximum ADC sample value), LED0 should be turned on; if the current reading is below the halfway point, turn LED0 off
 - Implement a button interrupt for push-button PA0 such that, when pressed, the interrupt handler should toggle the state of LED7; you may ignore debouncing

Draw a labelled flowchart showing how you could go about implementing all of the above, showing your logic clearly.

Solution: Open-ended, but the mark breakdown is generally:

Overall diagram and logic [3]

POT0 value to LCD [1]

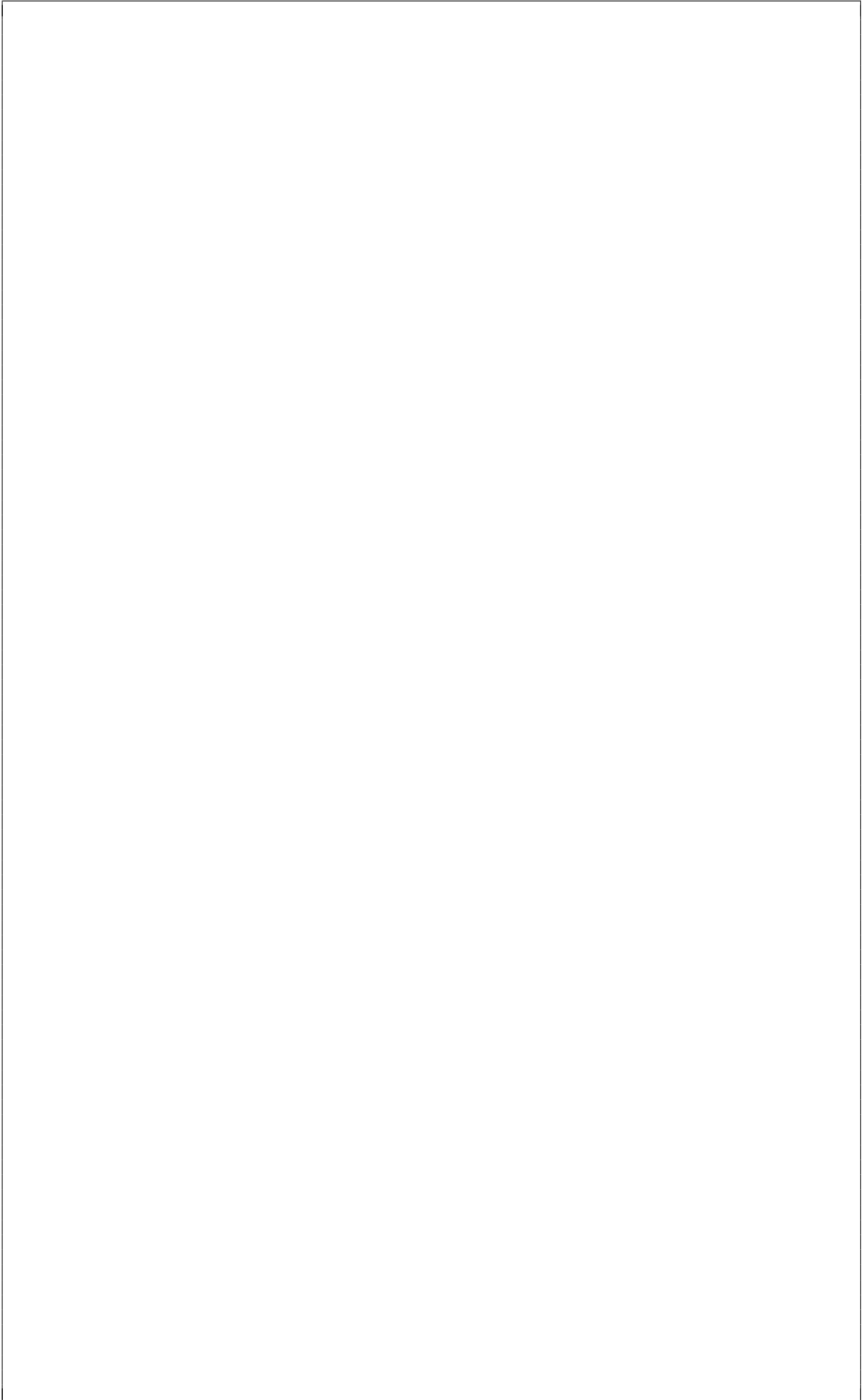
LED0 comparison logic [1]

Push-button interrupt toggling LED7; Interrupt Service Routine (ISR) separate from main flowchart [2]

Descriptions/labels [2]

The diagram should simply illustrate some kind of logic to sample POT0, output this value to the LCD, and compare the value to the halfway point, e.g., ADC value ≥ 2048 ; if exceeded, turn LED0 on, otherwise turn it off.

Additionally, the ISR should be separate from the main flowchart; whereas the main flowchart may start from "START" or "MAIN", the interrupt should start separately – perhaps with something like "ISR". This should then implement logic that checks the state of LED7 and toggles it; if High, it should be set Low, and if Low, it should be set High.



Part BEmpl ID:

--	--	--	--	--	--	--

Question:	6	7	8	9	10	Total
Points:	12	16	16	15	16	75
Score:						

Question 6: Multiple Choice [12 marks]

a. Among these options, which one does **not** constitute a part of the 16-bit register frame structure? (2)

- ☒ **Function Unit (FU)**
☐ Opcode
☐ Destination register (DR)
☐ Source register A (SA)

b. For a typical telephone line with a signal-to-noise ratio of 40 dB and an audio bandwidth of 3 kHz, what would be the maximum data rate? (4)

- ☐ 29,901 bps
☐ 30,002 bps
☒ **39,870 bps**
☐ 19,910 bps

c. "A band-limited analog signal that has been sampled can be perfectly reconstructed from an infinite sequence of samples if the sampling rate f_s exceeds $2f_{max}$ samples per second, where f_{max} is the highest frequency in the original signal". Which theory is this? (2)

- ☐ Shannon's theory
☒ **Nyquist's theory**
☐ Thevenin's theory
☐ Berlo's theory

d. Spatial computation is typical of programmers. True or false? (2)

- ☐ True
☒ **False**

e. A simple 2-select line multiplexer includes four AND gates and one OR gate. True or false? (2)

- ☒ **True**
☐ False

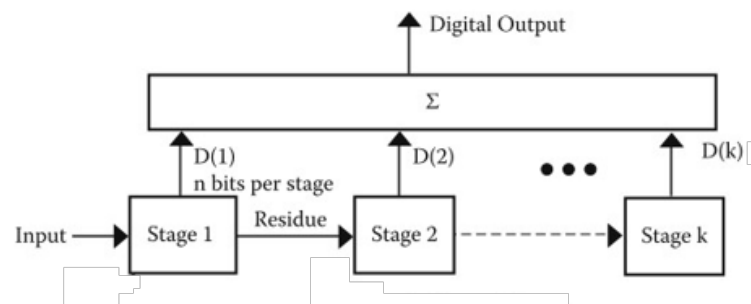
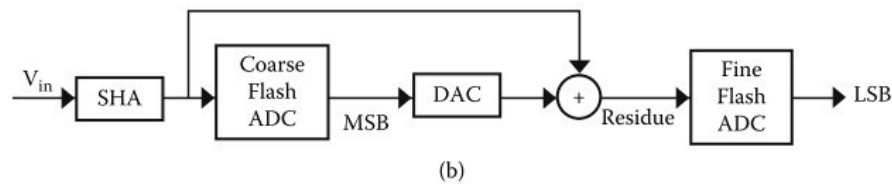
Question 7: ADCs and DACs [16 marks]

- a. Multistage and pipelined flash Analog-to-Digital Converters (ADCs) have some similarities in their operation. How would you differentiate the two? Use diagrams in your explanation. (10)

Solution: A multistage flash ADC uses a number of smaller flash ADCs to decide on a sample. The Sample-and-Hold (SAH) circuit sends the signal to the coarse flash ADC that converts and interprets the analogue signal for the MSB. The DAC then converts this back to analogue, which is then compared to the sampled signal and produces a residue that is passed into the fine flash ADC, resulting in the LSB.

On the other hand, the pipelined flash will run Stage 1 – which does a coarse estimate – and then Stage 2 runs and does a fine estimate, but simultaneously Stage 1 starts a coarse estimate on a new sample. The key aspect here is the SAH at each stage – essentially causing a smaller and smaller residual to move down the pipeline until the final LSB is decided.

Figures below.





- b. What would be the quantisation step size (in Volts) of a 16-bit ADC with a voltage range of 0 to 4.5 V? (2)

Solution: Quantisation step size (in Volts) = (Voltage Range) / (2^{res})
 Quantisation step size = $(4.5)/(2^{16}) = 4.5/65536 = 0.00006866455078125 \text{ V} = 68.66 \text{ } \mu\text{V}$

- c. For a 4-bit R2R ladder DAC: if the voltage output is 3.25 V, determine the equivalent digital input signal in binary. Assume a reference voltage of 5 V. (4)

Solution:

$$D = \frac{V_{\text{out}}}{V_{\text{ref}}} \times 2^n$$

where:

D is the digital input signal

V_{out} is the voltage output

V_{ref} is the reference voltage

n is number of bits in the DAC

Therefore:

$$D = \frac{3.25}{5} \times 2^4$$

$$D = 0.65 \times 16$$

$$D = 10.4 \approx 10$$

The 4-bit binary representation is thus:

$$D = 10_{10} = 1010_2$$

So, for the given 4-bit R2R ladder DAC, the equivalent digital input signal is 0b1010.

Question 8: Finite State Machines [16 marks]

Imagine you own a tech design startup and a client has asked you to design an automatic tea/coffee vending machine based on an FSM model. The customer has given you the following system requirements that you must adhere to:

1. The tea/coffee is initiated by the user inserting a R5 coin.
2. After inserting the coin, the user can either select "Coffee" or "Tea", or press "Cancel" to revoke the order and take back the coin. The order is automatically cancelled (and the coin is pushed back) if a 10-second timer expires.
3. If the user selects coffee or tea, coffee or tea should be correctly dispensed.

Answer the following questions.

- a. What are the states of the design?

(4)

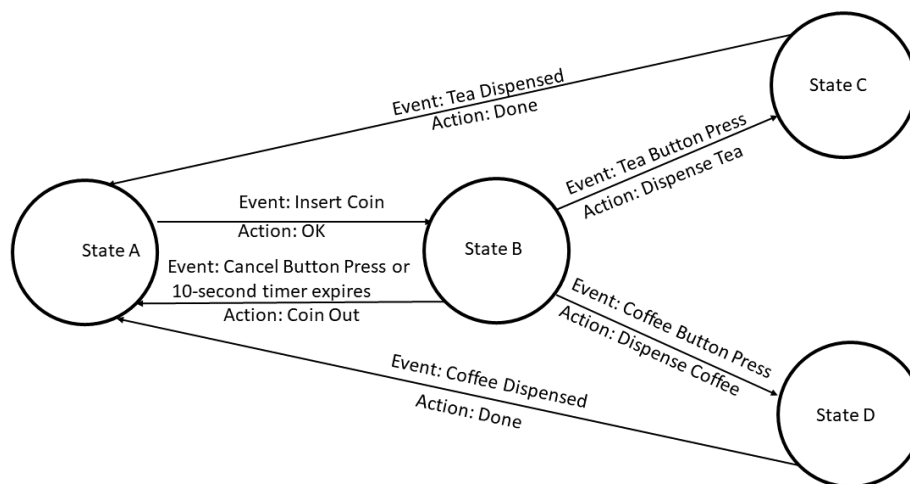
Solution:

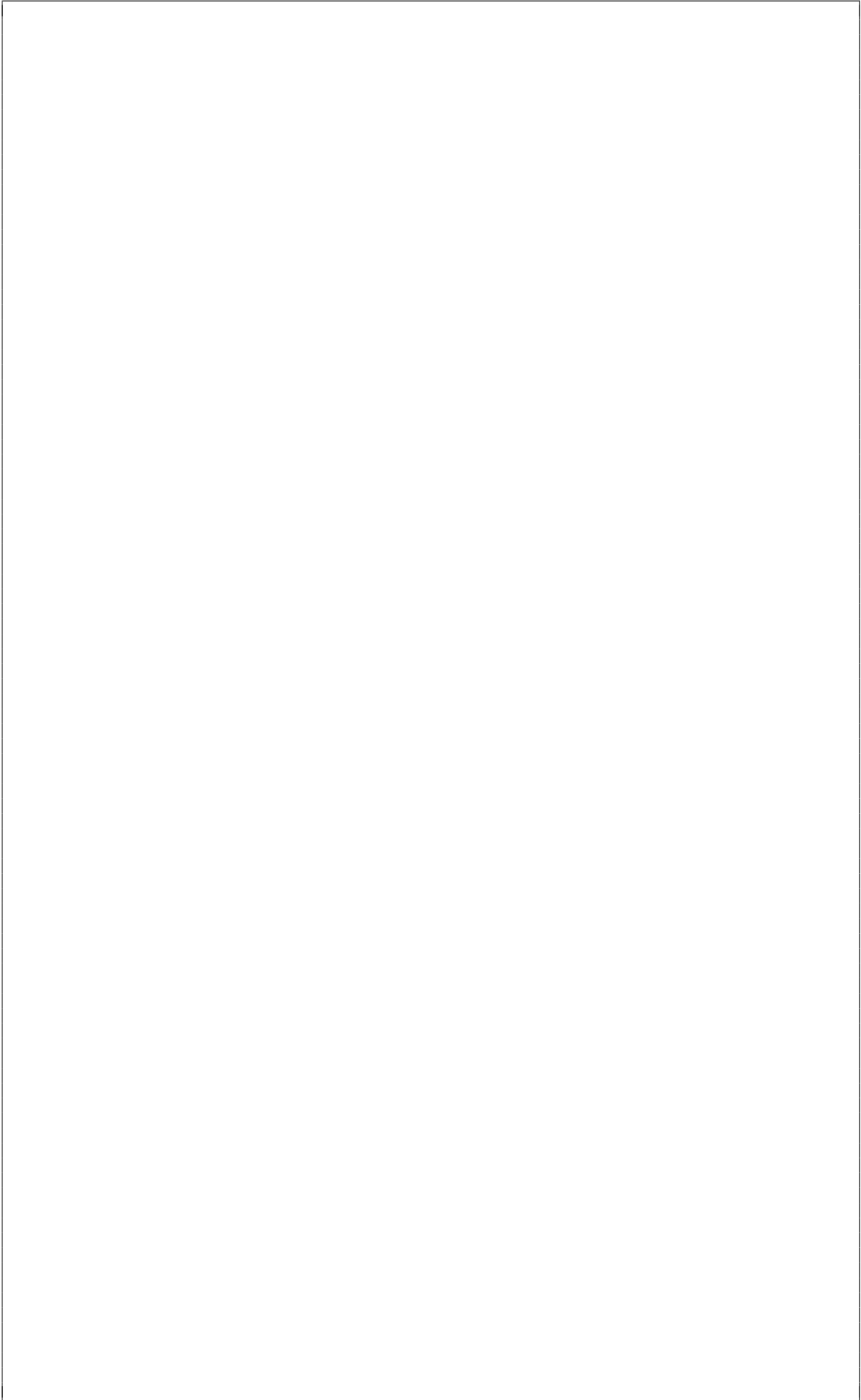
- State A: Wait for coin
- State B: Wait for user input
- State C: Dispense tea
- State D: Dispense coffee

- b. Draw the finite state model diagram showing all states and actions.

(12)

Solution: Sample figure below; can be arranged differently but should make the same logical sense.





Question 9: FPGAs and Verilog [15 marks]

- a. FPGAs are considered flexible and highly parallel programmable logic chips that are widely used in prototypes and final versions of embedded systems. List and briefly describe any four of the seven consecutive steps of the FPGA development flow. (8)

Solution: See below:

1. Design and HDL/RTL Coding:
Behavioural or Structural Description of Design
Writing VHDL, deciding I/O, formulating tests
2. RTL Simulation:
Functional Simulation
Verify Logic Model and Data Flow
View model-specified timing
3. Synthesis:
Translate Design into Device Specific Primitives
Optimisation to meet Area and Performance Constraints
4. Place and Route (PAR):
Map primitives to specific locations inside FPGA with reference to area performance constraints
Specify routing resources to use
5. Timing Analysis:
Verify performance specifications
Static timing analysis
6. Gate Level Simulation (GLS):
Timing simulation
Verify design will work on target platform
7. Program and test on hardware:
Generate bit file
Program target device
Activate the system

- b. Verilog is all about modules, ports, wires, and registers. Write a simple 2-input AND gate as a module to show the use of these ingredients; the module name should be your student number. (5)

Solution: See below:

```
module "Student-ID" (  
    input wire A,  
    input wire B,  
    output wire  
);  
    assign Y = A & B;  
endmodule
```

- c. Modules are built from other modules and/or built-in modules; these other modules are instantiated in the module you are implementing. Why do we give instances names? (2)

Solution: The main reason for putting in a name is for debugging purposes; Xilinx tends to assign instance names arbitrarily, but then you may get an error message such as “Cannot connect signals to XXYY01”, costing you time in trying to track down and figure out which gate is causing the problem.

Question 10: Verilog Simulation, CPU Architecture and Datapath [16 marks]

a. A 4-bit up-counter is implemented in the Verilog pseudocode shown below:

(9)

```
module counter (clk, reset, enable, count); // Define port types and directions
    input clk, reset, enable;
    output [3:0] count;
    reg [3:0] count;
    always @ (posedge clk)
    if (reset == 1'b1) begin
        count <= 0;
    end else if ( enable == 1'b1) begin
        count <= count + 1;
    end
endmodule
```

To test the functionality of the module and observe its behavior during simulation, write a testbench that initialises the input signals, toggles the clock signal, and monitors the values of **clk**, **reset**, **enable**, and **count** at different simulation time steps.

Solution: Solution code below:

```
// 4-bit Upcounter testbench
module counter_tb;
    reg clk, reset, enable;
    wire [3:0] count;

    counter U0 (        // instantiate the module
        .clk      (clk),.reset   (reset),.enable (enable),.count  (count));

    initial
    begin
        // monitor routine to keep printing out the pins we are interested in...
        $display("\t\ttime,\tclk,\treset,\tenable,\tcount");
        $monitor("%d,\tb,\tb,\tb,\td",$time, clk,reset,enable,count);
        // Now exercise the pins!!!
        clk = 0; reset = 0; enable = 0;
        #5 clk = !clk;           // The # says pause for x simulation steps
                                // The command just toggles the clock
        reset = 1;
        #5 clk = !clk;          // Let's just toggle it again for good measure
        reset = 0;              // Lower the reset line
        enable = 1;             // now start counting!!
        repeat (10) begin
            #5 clk = !clk;      // Let's just toggle it a few more times
        end
    end
endmodule
```



- b. The Function Unit (FU) carries out the computer operations and typically contains the Arithmetic Logic Unit (ALU) and the Control Unit (CU), i.e., $FU = ALU + CU$. What are the two main functions of the Control Unit? (2)

Solution: The two functions are as follows:

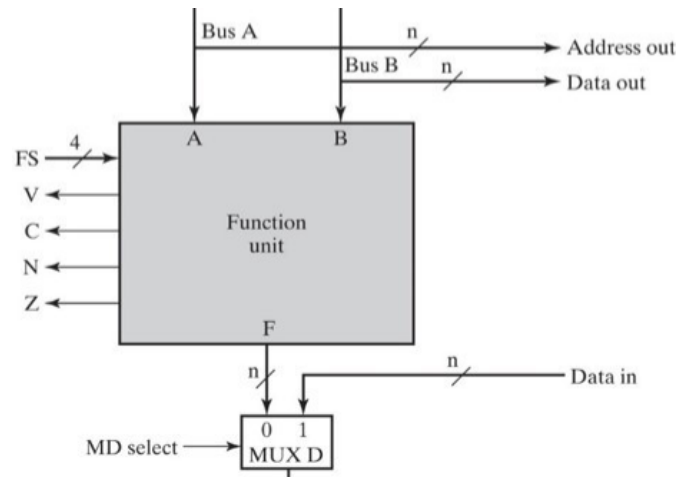
- The CU is the part of the CPU that directs the working of the processor.
- The CU coordinates the computer's memory, ALU and its input and output devices, making these parts respond to program instructions.

- c. With the aid of a diagram, explain the operation of the FU block in a single-cycle computer.

(5)

Solution: The FU has two input registers, A and B, upon which it operates. There is a Function Selector (FS) input used to choose the operation to perform. When this operation is performed, the result output is latched to F and updates the status flags V, C, N, and Z. The Memory Data (MD) select can bypass the FU if the instruction is loading from memory.

Sample figure below.



END OF EXAM

Appendix A: Verilog Cheat Sheet

Numbers and constants

Example: 4-bit constant 10 in binary, hex and in decimal: 4'b1010 == 4'ha -- 4'd10

(numbers are unsigned by default)

Concatenation of bits using {}

4'b1011 == {2'b10, 2'b11}

Constants are declared using parameter:

parameter myparam = 51

Operators

Arithmetic: and (+), subtract (-), multiply (*), divide (/) and modulus (%) all provided.

Shift: left (<<), shift right (>>)

Relational ops: equal (==), not-equal (!=), less-than (<), less-than or equal (<=), greater-than (>), greater-than or equal (>=).

Bitwise ops: and (&), or (|), xor (^), not (~)

Logical operators: and (&&) or (||) not (!) note that these work as in C, e.g. (2 && 1) == 1

Bit reduction operators: [n] n=bit to extract

Conditional operator: ? to multiplex result

Example: (a==1)? funcif1 : funcif0

The above is equivalent to:

((a==1) && funcif1)

|| ((a!=1) && funcif0)

Registers and wires

Declaring a 4 bit wire with index starting at 0:

wire [3:0] w;

Declaring an 8 bit register:

reg [7:0] r;

Declaring a 32 element memory 8 bits wide:

reg [7:0] mem [0:31]

Bit extract example:

r[5:2] returns 4 bits between pos 2 to 5 inclusive

Assignment

Assignment to wires uses the assign primitive outside an always block, e.g.:

assign mywire = a & b

Registers are assigned to inside an always block which specifies where the clock comes from, e.g.:

always@(posedge myclock)

cnt = cnt + 1;

Blocking vs. unblocking assignment <= vs. =

The <= assignment operator is non-blocking (i.e. if use in an always@(posedge) it will be performed on every positive edge. If you have many non-blocking assignments they will all be updated in parallel. The <= operator must be used inside an always block – you can't use it in an assign statement.

The blocking assignment operator = can be used in either an assign block or an always block. But it causes assignments to be performed in sequential order. This tends to result in slower circuits, so avoid using it (especially for synthesized circuits) unless you have to.

Case and if statements

Case and if statements are used inside an always block to conditionally update state. e.g.:

```
always @(posedge clock)
  if (add1 && add2) r <= r+3;
  else if (add2) r <= r+2;
  else if (add1) r <= r+1;
```

Note that we don't need to specify what happens when add1 and add2 are both false since the default behavior is that r will not be updated. Equivalent function using a case statement:

```
always @(posedge clock)
  case({add2,add1})
    2'b11 : r <= r+3;
    2'b10 : r <= r+2;
    2'b01 : r <= r+1;
    default: r <= r;
  endcase
```

Module declarations

Modules pass inputs, outputs as wires by default.

```
module ModName (
  output reg [3:0] result, // register output
  input [1:0] bitsin, input clk, inout bidirectnl );
  ... code ...
endmodule
```

Verilog Simulation / ISIM commands

```
$display ("a string to display");
$monitor ("like printf. Vals: %d %b", decv,bitv);
#100 // wait 100ns or simulation moments
$finish // end simulation
```

Appendix B: ARM Assembly Language Cheat Sheet

Memory access instructions

```
LDR Rd, [Rn]           ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off]      ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn]           ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off]      ; store 32-bit Rt to [Rn+off]
PUSH {Rt}              ; push 32-bit Rt onto stack
POP {Rd}               ; pop 32-bit number from stack into Rd

MOV{S} Rd, <op2>       ; set Rd equal to op2
MOV Rd, #iml6           ; set Rd equal to iml6, iml6 is 0 to 65535
MVN{S} Rd, <op2>       ; set Rd equal to -op2
```

Branch instructions

```
B label                ; branch to label Always
BEQ label              ; branch if Z == 1 Equal
BNE label              ; branch if Z == 0 Not equal
BCS label              ; branch if C == 1 Higher or same, unsigned ≥
BHS label              ; branch if C == 1 Higher or same, unsigned ≥
BCC label              ; branch if C == 0 Lower, unsigned <
BLO label              ; branch if C == 0 Lower, unsigned <
BMI label              ; branch if N == 1 Negative
BPL label              ; branch if N == 0 Positive or zero
BVS label              ; branch if V == 1 Overflow
BVC label              ; branch if V == 0 No overflow
BHI label              ; branch if C==1 and Z==0 Higher, unsigned >
BLS label              ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label              ; branch if N == V Greater than or equal, signed ≥
BLT label              ; branch if N != V Less than, signed <
BGT label              ; branch if Z==0 and N==V Greater than, signed >
BLE label              ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm                  ; branch indirect to location specified by Rm
BL label               ; branch to subroutine at label
BLX Rm                 ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd}, Rn, <op2> ; Rd=Rn&op2 (op2 is 32 bits)
ORR{S} {Rd}, Rn, <op2> ; Rd=Rn|op2 (op2 is 32 bits)
EOR{S} {Rd}, Rn, <op2> ; Rd=Rn^op2 (op2 is 32 bits)
BIC{S} {Rd}, Rn, <op2> ; Rd=Rn&(~op2) (op2 is 32 bits)
ORN{S} {Rd}, Rn, <op2> ; Rd=Rn|(~op2) (op2 is 32 bits)
LSR{S} Rd, Rm, Rs      ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n      ; logical shift right Rd=Rm>>n (unsigned)

ASR{S} Rd, Rm, Rs      ; arithmetic shift right Rd=Rm>>Rs (signed)
ASR{S} Rd, Rm, #n      ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs      ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n      ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd}, Rn, <op2> ; Rd = Rn + op2
ADD{S} {Rd}, Rn, #iml2 ; Rd = Rn + iml2, iml2 is 0 to 4095
SUB{S} {Rd}, Rn, <op2> ; Rd = Rn - op2
SUB{S} {Rd}, Rn, #iml2 ; Rd = Rn - iml2, iml2 is 0 to 4095
RSB{S} {Rd}, Rn, <op2> ; Rd = op2 - Rn
RSB{S} {Rd}, Rn, #iml2 ; Rd = iml2 - Rn
CMP Rn, <op2>          ; Rn - op2 sets the NZVC bits
MUL{S} {Rd}, Rn, Rm     ; Rd = Rn * Rm signed or unsigned
```

Notes

Ra Rd Rm Rn Rt represent 32-bit registers

value any 32-bit value: signed, unsigned, or address

{S} if S is present, instruction will set condition codes

#im12 any value from 0 to 4095

#im16 any value from 0 to 65535

{Rd,} if Rd is present Rd is destination, otherwise Rn

#n any value from 0 to 31

#off any value from -255 to 4095

label any address within the ROM of the microcontroller

op2 the value generated by <op2>